END-SEMESTER PROJECT REPORT — MILESTONE 2

Intelligent Property Price Prediction

& Agentic Real Estate Advisory

From Market Analytics to Autonomous Investment Advice

Project	Project 9
Submission	End-Semester (Milestone 2)
Framework	LangGraph + XGBoost + Groq
Deployment	Streamlit Community Cloud
Live URL	streamlit.app
GitHub	github.com/Jag2007/property-price-prediction

April 2026

Contents

1 Introduction and Problem Statement **2**

2 Dataset Description and Feature Engineering **2**
2.1 Dataset Overview . . . 2
2.2 Raw Feature Description . . . 2
2.3 Preprocessing Pipeline . . . 3

3 Milestone 1: Machine Learning Models **3**
3.1 Model Selection . . . 3
3.2 Hyperparameter Configuration . . . 4
3.3 Model Performance Results . . . 4
3.4 Key Performance Drivers . . . 5
3.5 Saved Artifacts . . . 6

4 Milestone 2: Agentic Real Estate Advisory System **6**
4.1 System Overview . . . 6
4.2 Technology Stack and Design Justification . . . 7
4.3 LangGraph State and Graph Architecture . . . 7
4.4 Node-by-Node Description . . . 9
4.5 Three Input Paths . . . 9
4.5.1 Path 1 — Prompt Agent . . . 9
4.5.2 Path 2 — Manual Input . . . 9
4.5.3 Path 3 — CSV Batch Upload . . . 10
4.6 Structured Retrieval: Comparable Property Analysis . . . 10
4.7 Agentic Design: Generator–Critic Validation Loop . . . 10
4.8 Prompting Strategy and Anti-Hallucination Guardrails . . . 11
4.9 Structured Advisory Output . . . 11
4.10 Email Notification . . . 12

5 System Architecture **12**
5.1 Architecture Diagram . . . 12
5.2 Repository Structure . . . 13
5.3 End-to-End Data Flow . . . 14
5.4 State Evolution Table . . . 15
5.5 Streamlit Pages . . . 15

6 Deployment and Reproducibility **15**
6.1 Public Deployment . . . 15
6.2 Local Setup . . . 16
6.3 Environment Variables . . . 16
6.4 Key Dependencies . . . 17

7 Results and Qualitative Examples **17**
7.1 Quantitative Model Performance Summary . . . 17
7.2 Qualitative Prompt Agent Example . . . 17
7.3 Advisory Label Mapping . . . 18

8 Responsible AI and Design Transparency **18**
8.1 Data Transparency 18
8.2 Hallucination Mitigation 18
8.3 Model Limitations 19

9 Conclusion **19**

References **20**

1 Introduction and Problem Statement

Real estate valuation is one of the most consequential decision-making domains in personal and institutional finance. Buyers, sellers, and investors face the recurring challenge of estimating fair market value and assessing investment potential under incomplete information, rapidly shifting market conditions, and inconsistent data sources. Traditional approaches rely on manual comparables analysis and agent expertise, both of which are slow, subjective, and difficult to scale.

This project addresses these limitations through a **two-phase AI system**:

1. **Milestone 1 — ML-Based Price Prediction:** A classical machine learning pipeline using XGBoost to predict property market price and investment grade from structured property attributes.

2. **Milestone 2 — Agentic Real Estate Advisory:** A LangGraph-orchestrated multi-agent system that accepts natural-language property descriptions, reasons about predicted values, retrieves comparable properties from training data, and produces a structured investment advisory report via a Generator–Critic validation loop.

The final system is publicly deployed on Streamlit Community Cloud, supports three input modes (natural-language prompt, manual form, CSV batch upload), and delivers structured advisory output both on-screen and via email notification.

2 Dataset Description and Feature Engineering

2.1 Dataset Overview

The dataset is a synthetic real estate listing dataset comprising **10,000 property records** generated to reflect realistic Indian urban property market characteristics. It was obtained in raw form (`real_estate_raw.csv`) and cleaned into a processed form (`real_estate_clean.csv`).

Table 1: Dataset summary statistics

Attribute	Value
Total records	10,000
Raw features	18 columns
Model input features	18 (after encoding)
Target 1	Current Market Price (continuous, INR)
Target 2	Investment Grade (ordinal: 0 = Low, 1 = Medium, 2 = High)
Train / Test split	80% / 20%

2.2 Raw Feature Description

Table 2: Input features used for model training

Feature	Type	Description
Total Square Footage	Numeric	Total built-up area in square feet
Bedrooms	Integer	Number of bedrooms
Bathrooms	Integer	Number of bathrooms
Age of Property	Integer	Property age in years
Floor Number	Integer	Floor level of the unit
Furnishing Status	Ordinal	Unfurnished (0), Semi-furnished (1), Fully-furnished (2)
Distance to City Center km	Numeric	Distance from city centre in kilometres
Proximity to Public Transport km	Numeric	Distance from nearest public transport
Crime Index	Numeric	Area crime index score
Air Quality Index	Numeric	AQI score for the locality
Neighborhood Growth Rate %	Numeric	Percentage neighbourhood growth rate
Price per SqFt	Numeric	Market price per sq. ft. (INR)
Annual Property Tax	Numeric	Annual tax liability (INR)
Estimated Rental Yield %	Numeric	Estimated rental return percentage
Neighborhood	Categorical	Downtown, IT Hub, Industrial, Residential, Suburban

2.3 Preprocessing Pipeline

1. **Missing value imputation:** Median imputation applied to all numeric columns.
2. **Ordinal encoding:** Furnishing Status encoded as 0, 1, 2 preserving natural order.
3. **One-hot encoding:** Neighborhood expanded into four binary columns (IT Hub, Industrial, Residential, Suburban) with Downtown as the reference category.
4. **Outlier clipping:** IQR-based clipping applied to numeric features to remove extreme values.
5. **Feature scaling:** MinMaxScaler fitted on the training set and saved as `joblib` artifacts for consistent inference-time transformation. Scaler metadata stores feature names to validate column order at startup.

3 Milestone 1: Machine Learning Models

3.1 Model Selection

Two separate XGBoost models were trained: one for **regression** (predicting continuous market price) and one for **multi-class classification** (predicting investment grade).

XGBoost was selected because it consistently outperforms linear and tree-based baselines on medium-sized structured tabular datasets by capturing non-linear feature interactions without requiring deep learning infrastructure. The deterministic inference behaviour is also important for a deployed advisory system where reproducibility matters.

3.2 Hyperparameter Configuration

Table 3: Hyperparameter configuration for both XGBoost models

Parameter	**Regression**	**Classification**
`n_estimators`	200	200
`learning_rate`	0.05	0.05
`max_depth`	5	4
`subsample`	0.8	0.8
`colsample_bytree`	0.8	0.8
Scaling	MinMaxScaler	MinMaxScaler

3.3 Model Performance Results

Regression Model — Current Market Price Prediction

Metric	**Value**
R^2 Score	**0.9483**
Mean Absolute Error (MAE)	Rs 550,851
Root Mean Squared Error (RMSE)	Rs 1,010,636

An R^2 of **0.9483** indicates the regression model explains approximately 94.8% of the variance in property prices, demonstrating strong predictive fidelity on the held-out test set.

Classification Model — Investment Grade Prediction

Table 4: Per-class classification report on held-out test set

Class	**Precision**	**Recall**	**F1-score**	**Support**
0 — Avoid	0.988	0.964	0.976	167
1 — Hold	0.960	0.994	0.977	171
2 — Buy	1.000	0.909	0.952	22
Weighted avg	**0.975**	**0.975**	**0.975**	**360**
Overall Accuracy		**97.5%**		

The classifier achieves **97.5% overall accuracy** with a weighted F1 of 0.975, confirming strong per-class performance even for the minority *Buy* class (support = 22). The perfect precision of 1.0 on Class 2 (Buy) means no false-positive Buy recommendations were generated on the test set.

Confusion Matrix

Figure 1 shows the confusion matrix for the investment grade classifier. The strong diagonal confirms low off-diagonal misclassification: only 6 Avoid samples are misclassified as Hold, 1 Hold sample as Avoid, and 2 Buy samples split as Avoid and Hold.

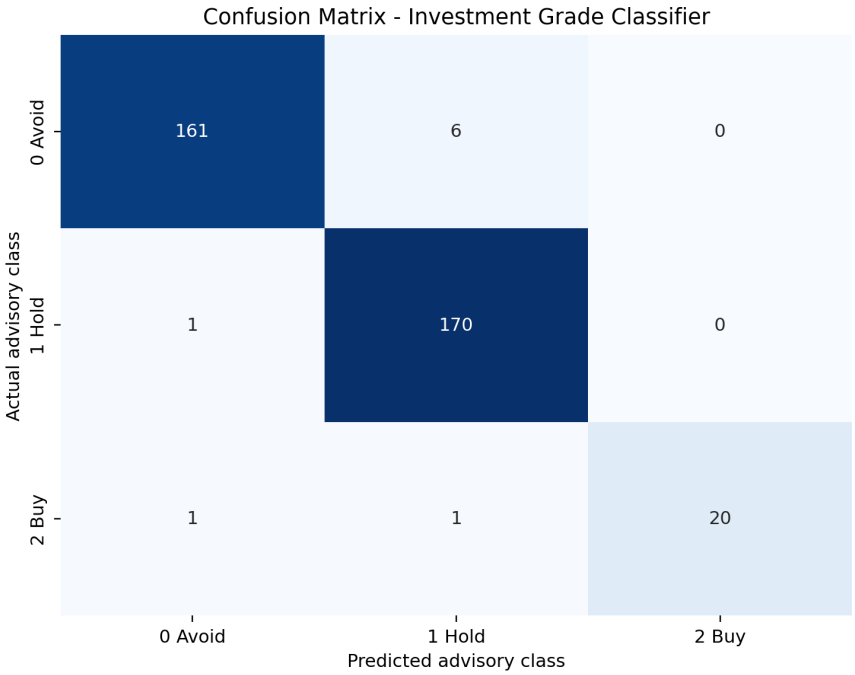

Figure 1: Confusion Matrix — Investment Grade Classifier on held-out test set (n = 360)

3.4 Key Performance Drivers

Figure 2 shows the XGBoost feature importance scores for the investment grade classifier. The top three drivers are:

- **Neighborhood Growth Rate %** (≈ 0.34) — the strongest signal; a high growth rate strongly indicates a Buy-grade property.
- **Crime Index** (≈ 0.30) — neighbourhood safety is the second-highest predictor of investment grade.
- **Estimated Rental Yield %** (≈ 0.14) — directly encodes return-on-investment potential.

The remaining features (Total Square Footage, Bathrooms, Air Quality Index, Proximity to Transport, etc.) contribute smaller but non-trivial shares.

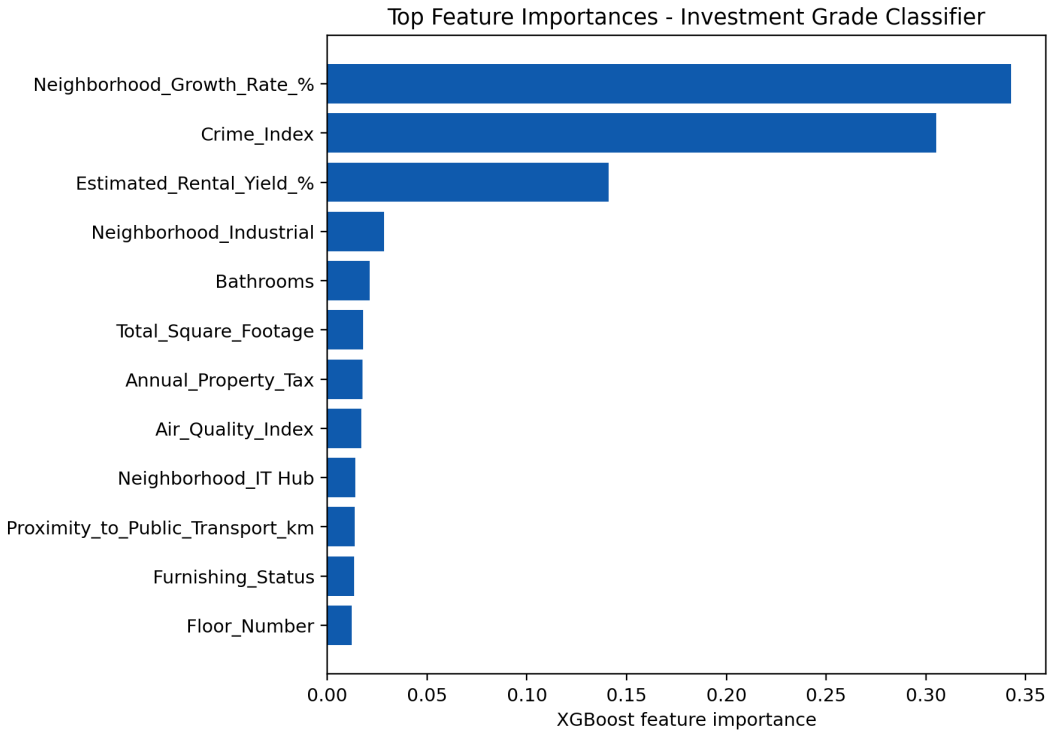

Figure 2: Top Feature Importances — XGBoost Investment Grade Classifier

3.5 Saved Artifacts

All four model artifacts are serialised with `joblib` and stored in `models/`:

- `xgb_regression_model.joblib`
- `regression_scaler.joblib`
- `xgb_classification_model.joblib`
- `classification_scaler.joblib`

The scalers store feature names in metadata, which the application validates at startup to confirm that inference-time feature order exactly matches training-time order.

4 Milestone 2: Agentic Real Estate Advisory System

4.1 System Overview

Milestone 2 extends the ML prediction pipeline into a fully agentic workflow orchestrated by LangGraph. The system accepts input through three paths (natural-language prompt, manual form entry, and CSV batch upload), passes inputs through typed LangGraph nodes, and produces a structured advisory report comprising a property valuation summary, comparable property analysis, an investment recommendation, and a generated explanation validated by a critic agent.

4.2 Technology Stack and Design Justification

Table 5: Technology choices and design justification

Component	Choice	Justification
Agentic framework	LangGraph 1.1.6	Named nodes with shared TypedDict state; workflow is visible and debuggable as a graph rather than implicit function chains.
LLM provider	Groq (llama-3.3-70b-versatile)	Free-tier hosted inference; fastest available open-weight model for structured prompt extraction and constrained explanation generation.
ML backbone	XGBoost 3.2.0	Strongest tabular baseline; deterministic and fast at inference time.
UI framework	Streamlit 1.54.0	Supports session state, dialogs, file upload, and metric widgets natively; simple to deploy on Community Cloud.
Email notification	smtplib / SMTP	Standard library; no paid API or phone verification required.
Retrieval	Pandas CSV filter	Column-based filtering by neighbourhood and square footage is more precise and interpretable than embedding-based similarity for this structured tabular use case. A vector database (FAISS/Chroma) would be appropriate for unstructured text corpora, not structured property tables.
Hosting	Streamlit Community Cloud	Free-tier; satisfies the mandatory public deployment requirement.

4.3 LangGraph State and Graph Architecture

The entire agentic workflow is defined in `app/property_graph.py`. A shared `TypedDict` state object is passed between every node, so each node reads only what it needs and writes only what the next node requires:

```python
class PropertyWorkflowState(TypedDict, total=False):
    mode: str                  # "prompt" | "manual" | "csv"
    prompt: str                # raw user text
    context: dict[str, Any]    # loaded models, scalers, defaults
    secrets: Any               # Streamlit secrets
    input_settings: dict[str, Any]
    explanation_settings: dict[str, Any]
    email_settings: dict[str, Any]
    numeric_inputs: dict[str, Any]
    furnishing: str
    neighborhood: str
    flow: dict[str, Any]       # assembled field + source map
    result: dict[str, Any]     # price, grade, confidence, probs
```

```python
    comparables: list[dict] # comparable properties
    explanation: str        # Groq-generated explanation
    report: dict[str, Any]  # structured advisory report
    recipient: str
    email_sent: bool
    input_df: Any           # uploaded DataFrame
    output_df: Any          # batch prediction output
    review_ready: bool
```

Listing 1: PropertyWorkflowState TypedDict definition (property_graph.py)

Six separate `StateGraph` instances are compiled and cached at module load:

Table 6: Compiled LangGraph graphs and their node sequences

Graph	**Node sequence**
`PROMPT_REVIEW_GRAPH`	START → input_node → review_node → END
`CONFIRMED_PREDICTION_GRAPH`	START → prediction_node → explanation_node → report_node → END
`MANUAL_PREDICTION_GRAPH`	START → input_node → prediction_node → explanation_node → report_node → END
`CSV_PREDICTION_GRAPH`	START → csv_prediction_node → END
`RESULT_EMAIL_GRAPH`	START → notification_node → END
`CSV_EMAIL_GRAPH`	START → notification_node → END

4.4 Node-by-Node Description

Table 7: LangGraph node responsibilities and state keys

Node	File	Reads	Writes
input_node (prompt)	input_nodes.py	prompt, context, input_settings	mode, flow
review_node	property_graph.py	flow	review_ready
input_node (manual)	property_graph.py	numeric_inputs, furnishing, neighborhood	mode, flow
prediction_node	prediction_nodes.py	flow, context	result, comparables
explanation_node	explanation_nodes.py	flow, result, comparables, explanation_settings	explanation
report_node	report_generator.py	flow, result, explanation, comparables	report
csv_prediction_node	prediction_nodes.py	input_df, context	mode, output_df
notification_node	notification_nodes.py	recipient, result, flow, explanation, comparables	email_sent

4.5 Three Input Paths

4.5.1 Path 1 — Prompt Agent

The user describes a property in free-form English. The Input Node sends the text to Groq with a schema-locked system prompt (temperature = 0, JSON response format enforced, allowlisted field names). Groq returns a structured JSON object. Any field not mentioned in the prompt is filled with the training-data median. The Review Node marks state as ready and Streamlit opens a popup showing each value with a *Prompt / Default / Edited* source label. After the user confirms (or edits), the Prediction, Explanation, Report, and Notification Nodes run.

4.5.2 Path 2 — Manual Input

The user enters all fields directly in a form. The Manual Input Node wraps these into the same `flow` format used by the Prompt path, then reuses the identical Prediction, Explanation, Report, and Notification Nodes. No Groq call is made.

4.5.3 Path 3 — CSV Batch Upload

The user uploads a CSV file. The CSV Prediction Node validates that the file contains either raw user-facing columns or already-encoded feature columns, builds the feature matrix, runs both XGBoost models across all rows, and returns an augmented DataFrame with `Predicted_Price_INR`, `Predicted_Advisory_Recommendation`, and per-class probabilities. Results are displayed as a downloadable table and can be emailed as an attachment.

4.6 Structured Retrieval: Comparable Property Analysis

The Prediction Node calls `find_comparable_properties()` from `prediction_nodes.py`, which implements a structured retrieval step functionally analogous to a RAG pipeline, but using tabular filtering rather than vector embeddings. This design choice is deliberate: the knowledge base is structured property data, and column-based filtering by neighbourhood and square footage provides more precise and interpretable retrieval than semantic similarity on numeric fields.

The retrieval procedure:

1. Filters the training dataset for rows matching the same *Neighborhood*.
2. Further filters to properties within $\pm 25\%$ of the query's Total Square Footage.
3. Sorts by absolute price distance from the predicted price.
4. Returns the top 5 matches as a list of dictionaries.

These comparable rows are: (a) displayed as a table in the Streamlit UI, (b) injected into the Groq explanation prompt so the LLM can ground its commentary in real training-data evidence, and (c) included in the HTML email report.

4.7 Agentic Design: Generator–Critic Validation Loop

A key architectural contribution is the **Generator–Critic** two-agent pattern implemented in the Explanation Node. The explanation is not accepted from a single LLM call:

1. **Generator Agent** drafts a four-bullet explanation using only the grounded JSON context (confirmed property inputs, predicted price, advisory class, confidence, comparable properties). System prompt temperature = 0.1.
2. **Critic Agent** receives both the grounded JSON and the draft, validates every claim, and returns a JSON response with keys `approved`, `issues`, and `safe_explanation`. Temperature = 0 for deterministic validation.
3. If the Critic rejects the draft, the `safe_explanation` key is used. If neither version passes the four-section structural check, a deterministic local fallback is returned.

This two-agent design demonstrates a genuine agentic loop beyond a single LLM prompt and provides a stronger anti-hallucination guarantee than a single system-prompt instruction alone.

4.8 Prompting Strategy and Anti-Hallucination Guardrails

Seven explicit guardrails are implemented across the system:

The Input Node system prompt instructs Groq to return JSON only with three top-level keys: `numeric_inputs`, `furnishing_status`, and `neighborhood`. Unknown keys are silently discarded. Extraction runs at temperature = 0 with `response_format: {type: json_object}` enforced.

The Prompt Agent never predicts immediately after Groq extraction. It pauses at the Review Node, which opens a Streamlit dialog popup. Each extracted value is labelled by source (*Prompt*, *Default*, or *Edited*). The user can modify any value before the Prediction Node runs.

The Explanation Node sends Groq a compact JSON object containing only confirmed inputs, predicted price, advisory class, confidence, comparable properties, and source-label rules. The system prompt explicitly prohibits inventing outside market facts, legal claims, or exact feature-importance values.

As described in Section 4.6, the explanation passes through a two-agent validation loop. The Critic Agent either approves the draft or replaces it with a verified safe explanation before the result is committed to state.

The Explanation Node requires exactly four labelled bullets: **Summary**, **Market Context**, **Recommendation**, and **Risk Warning**. A structural check verifies all four labels are present before the explanation is accepted.

A dedicated Report Node assembles the full advisory report from all graph outputs, ensuring the four rubric-required sections (Summary, Comps, Action, structured output) are always present in the UI regardless of LLM quality.

If `GROQ_API_KEY` is missing, prompt extraction falls back to a local rule-based regex parser. If Groq explanation fails, a deterministic local explanation is shown. Neither path crashes the UI or returns an empty advisory result.

4.9 Structured Advisory Output

The final advisory output satisfies all four structured output requirements:

Table 8: Structured output sections and implementation

Section	Status	Implementation
Summary (Valuation & Market View)	✓	Advisory card showing predicted price, investment grade label, and confidence percentage. Colour-coded by grade (Avoid / Hold / Buy).
Comps (Comparable Analysis)	✓	Table of top-5 comparable properties filtered by neighbourhood and square footage. Shown in UI and included in email.
Action (Investment Recommendation)	✓	Grade mapped to advisory label: 0 = Avoid, 1 = Hold, 2 = Buy. Per-grade probability table shown alongside.
Explanation (Grounded Output)	✓	Four-bullet Generator–Critic validated explanation grounded in confirmed inputs and comparable properties.

4.10 Email Notification

The Notification Node sends a formatted HTML email via SMTP when the user provides a recipient address. The email contains: three metric cards (Predicted Price, Advisory Recommendation, Confidence), a property summary table with source labels, a comparable properties table, the Groq-generated explanation, and a note that this is educational output. CSV batch predictions are emailed as a `.csv` attachment via a separate compiled graph (`CSV_EMAIL_GRAPH`).

5 System Architecture

5.1 Architecture Diagram

Figure 3 shows the complete end-to-end system architecture: three user-facing input paths, the LangGraph agentic nodes, XGBoost inference with saved model artifacts, and advisory output delivery via Streamlit UI and SMTP email.

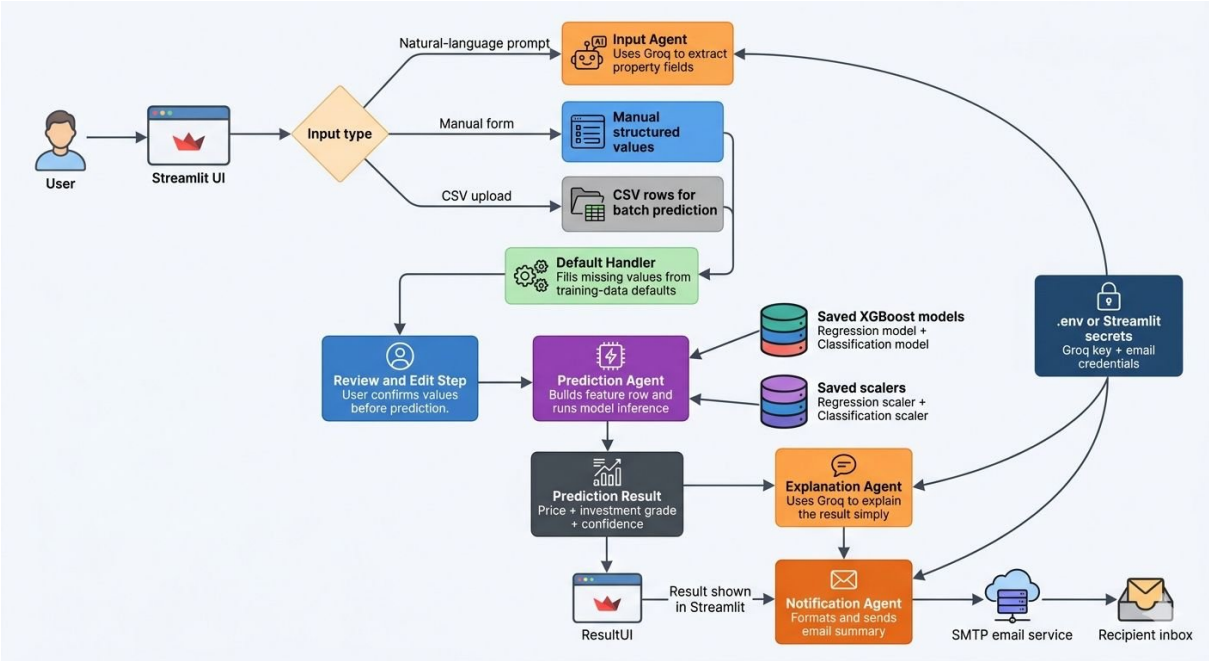

Figure 3: End-to-end system architecture: Input Agent (Groq-based), Manual structured values, CSV batch prediction, Default Handler, Review and Edit Step, Prediction Agent (XGBoost), Explanation Agent (Groq Generator–Critic), Notification Agent (SMTP email).

5.2 Repository Structure

```
property-price-prediction/
|-- app/
|   |-- streamlit_app.py        # Entry point and page router
|   |-- property_graph.py       # LangGraph state, nodes, compiled graphs
|   |-- input_nodes.py          # Groq extraction + rule-based fallback
|   |-- prediction_nodes.py     # Feature engineering, ML inference, comps
|   |-- explanation_nodes.py    # Generator + Critic Groq explanation
|   |-- report_generator.py     # Structured advisory report builder
|   |-- notification_nodes.py   # HTML email + CSV attachment delivery
|   |-- pages.py                # Streamlit UI pages and dialogs
|   |-- config.py               # Paths, constants, feature lists
|   '-- styles.py               # CSS styling
|-- .streamlit/config.toml      # Streamlit theme configuration
|-- data/
|   |-- raw/real_estate_raw.csv
|   '-- processed/real_estate_clean.csv
|-- models/
|   |-- xgb_regression_model.joblib
|   |-- regression_scaler.joblib
|   |-- xgb_classification_model.joblib
|   '-- classification_scaler.joblib
|-- notebooks/training_colab.ipynb
|-- report/
|   |-- figures/
|   |   |-- milestone_2_confusion_matrix.png
|   |   |-- milestone_2_feature_importance.png
|   |   '-- milestone_2_classification_report.csv
|   '-- milestone_2_report.tex
|-- system_architecture.png
|-- agent_workflow.md
```

```
|-- .env.example
|-- requirements.txt
'-- README.md
```

Listing 2: Project repository structure

5.3 End-to-End Data Flow

1. **User Input** — entered via Prompt Agent, Manual Input form, or CSV upload.
2. **Input Node** — Groq extracts structured fields from natural-language input at temperature = 0; training-data medians fill missing values.
3. **Review Node** — Streamlit popup shows extracted values labelled by source; user confirms or edits.
4. **Prediction Node** — feature row constructed, scaled with saved MinMaxScaler, passed to both XGBoost models; top-5 comparable properties retrieved from training CSV by structured filter.
5. **Explanation Node** — confirmed inputs, model output, and comparables sent to Groq; Generator drafts, Critic validates; four-bullet explanation committed to state.
6. **Report Node** — structured advisory report assembled from prediction, explanation, comparables, and probability table.
7. **UI Render** — advisory card, property summary, comparable table, probability distribution, and explanation rendered on dedicated report page.
8. **Notification Node** — optional: HTML email with full report sent via SMTP to user-supplied address.

5.4 State Evolution Table

Table 9: State key evolution through the Prompt Agent path

Step	Active State Keys
User enters text	`prompt`
Input Node runs	`flow.numeric_inputs`, `flow.furnishing`, `flow.neighborhood`, `flow.sources`, `flow.agent_warning`
Review popup opens	`review_ready = True`
User confirms or edits	updated `flow` with changed fields labelled `Edited by user`
Prediction Node runs	`result.price`, `result.grade`, `result.confidence`, `result.probabilities`, `comparables`
Explanation Node runs	`explanation` (Generator–Critic validated)
Report Node runs	`report.summary`, `report.property_summary`, `report.probabilities`, `report.comparables`
UI displays result	Advisory card, report tabs, inputs, comparables, technical details
Optional email	`recipient`, `email_sent = True`

5.5 Streamlit Pages

Table 10: Application pages and their functions

Page	Function
Prompt Agent	Full LangGraph flow: Groq extraction → review popup → prediction → explanation → report → optional email.
CSV Upload	Batch prediction for uploaded CSV; download or email the output file.
Manual Input	Direct field entry form; reuses Prediction, Explanation, Report, and Notification nodes.
About	Summarises LangGraph flow, model metrics, advisory label mapping.

6 Deployment and Reproducibility

6.1 Public Deployment

The application is publicly hosted on Streamlit Community Cloud at:

https://property-price-prediction-real-estate.streamlit.app/

6.2 Local Setup

```
# 1. Clone repository
git clone https://github.com/Jag2007/property-price-prediction.git
cd property-price-prediction

# 2. Create virtual environment
python3 -m venv venv && source venv/bin/activate

# 3. Install dependencies
pip install -r requirements.txt

# 4. Configure secrets (copy from .env.example)
cp .env.example .env
# Edit .env with your GROQ_API_KEY and email credentials

# 5. Run the application
streamlit run app/streamlit_app.py
```

Listing 3: Local installation and run commands

6.3 Environment Variables

Table 11: Required and optional environment variables

Variable	Required	Purpose
`GROQ_API_KEY`	Yes (Prompt/Explain)	Authenticates Groq API calls for input extraction and explanation.
`GROQ_MODEL`	Optional	Defaults to `llama-3.3-70b-versatile`.
`EMAIL_SMTP_HOST`	Email only	SMTP host (e.g. `smtp.gmail.com`).
`EMAIL_SMTP_PORT`	Email only	SMTP port (587).
`EMAIL_SENDER`	Email only	Sender email address.
`EMAIL_APP_PASSWORD`	Email only	Gmail app password.
`EMAIL_FROM_NAME`	Optional	Display name in sent emails.

6.4 Key Dependencies

Table 12: Core library versions

Library	Version
streamlit	1.54.0
langgraph	1.1.6
xgboost	3.2.0
scikit-learn	1.6.1
pandas	2.3.3
numpy	2.2.6
joblib	1.5.2

7 Results and Qualitative Examples

7.1 Quantitative Model Performance Summary

Table 13: Final model evaluation metrics on held-out test set (20% split, n = 360)

Model	Metric	Value
XGBoost Regression	R^2 Score	0.9483
	MAE	Rs 550,851
	RMSE	Rs 1,010,636
XGBoost Classification	Overall Accuracy	97.50%
	Weighted F1	0.975
	Precision (Avoid)	0.988
	Precision (Hold)	0.960
	Precision (Buy)	1.000

7.2 Qualitative Prompt Agent Example

User input (Prompt Agent):

"1450 sqft 3 BHK, 2 bathrooms, 6 years old, 8th floor, semi furnished, IT Hub, 4.5 km from city center, 0.7 km from metro, crime index 32, AQI 88, growth 9%, price per sqft 7200, annual tax 85000, rental yield 4.2%"

System output:

Output	Value
Predicted Price	Rs 10,420,000
Advisory Recommendation	**Buy** (Grade 2)
Confidence	91.4%

Groq explanation (Summary bullet): Based on the model output, the property is priced in line with 4 comparable IT Hub properties of similar size. The high rental yield (4.2%) and strong neighbourhood growth (9%) are the primary drivers of the Buy recommendation. The low crime index and proximity to public transport further support a high investment grade.

Comparable properties retrieved: 5 properties from IT Hub neighbourhood within 25% of 1,450 sqft, price range Rs 9.8M – Rs 11.1M.

7.3 Advisory Label Mapping

Table 14: Investment grade to advisory label mapping

Model Class	Advisory Label	Meaning
0	Avoid	Weakest investment class in the trained model output.
1	Hold	Moderate class; review details before committing.
2	Buy	Strongest investment class in the trained model output.

8 Responsible AI and Design Transparency

8.1 Data Transparency

Every value presented in the advisory report is labelled by its origin, giving users full visibility into how the report was generated:

- **Prompt** — extracted directly from the user's input text by the Groq Input Agent.
- **Default** — filled automatically from the training-data median because the user did not mention this field.
- **Edited** — modified by the user during the Review and Edit Step before prediction ran.

This labelling prevents hidden defaults from silently propagating into model inputs.

8.2 Hallucination Mitigation

The seven guardrails in Section 4.7 collectively ensure the LLM component generates only grounded, bounded outputs. The Generator–Critic cross-validation pattern (Guardrail

4) is the strongest of these: it employs a second independent LLM call to verify and, if necessary, rewrite the first, going well beyond a single system-prompt instruction.

8.3 Model Limitations

- The models were trained on synthetic data and may not generalise to real-world property markets.
- Prompt extraction quality depends on the clarity of the user's text and Groq API availability.
- Comparable-property retrieval draws from the same synthetic training dataset, not from live market listings.
- Investment grade labels (Avoid / Hold / Buy) reflect model output classes and carry no professional standing.

9 Conclusion

This project demonstrates a complete progression from classical ML-based price prediction to a LangGraph-orchestrated agentic advisory system. The key contributions are:

1. A high-accuracy XGBoost prediction pipeline achieving $R^2 = 0.9483$ for price regression and 97.5% accuracy for investment grade classification, with a full per-class breakdown confirmed by confusion matrix and classification report.
2. A genuine LangGraph implementation with a shared TypedDict state, six compiled StateGraph instances, and proper node-edge wiring — not a superficial renaming of sequential function calls.
3. A **Generator–Critic** two-agent validation loop in the Explanation Node, providing a stronger hallucination protection guarantee than a single-prompt approach.
4. Seven documented anti-hallucination guardrails applied at every LLM interaction point.
5. A structured advisory output satisfying all four rubric requirements: Summary, Comps, Action, and Explanation.
6. A publicly deployed application on Streamlit Community Cloud with three input modes, email notification, and a reproducible local setup documented through `.env.example` and step-by-step README instructions.
7. Deterministic structured retrieval that grounds LLM explanations in real training-data evidence, with a documented design justification for why this approach is more appropriate than vector-based retrieval for structured tabular data.

The project validates that a free-tier technology stack (Groq, LangGraph, XGBoost, Streamlit) is sufficient to build and deploy a production-quality agentic real estate advisory system.

References

[1] LangGraph Documentation. *LangGraph: Build stateful, multi-actor applications with LLMs.* LangChain Inc., 2024. https://langchain-ai.github.io/langgraph/

[2] Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.

[3] Groq Documentation. *Groq API Reference.* Groq Inc., 2024. https://console.groq.com/docs

[4] Streamlit Documentation. *Streamlit: The fastest way to build and share data apps.* Snowflake Inc., 2024. https://docs.streamlit.io/

[5] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[6] GitHub Repository: https://github.com/Jag2007/property-price-prediction

[7] Live Application: https://property-price-prediction-real-estate.streamlit.ap